

Report for assignment 4

This is a template for your report. You are free to modify it as needed. It is not required to use markdown for your report either, but the report has to be delivered in a standard, cross-platform format.

Project

Name: jackson-databind

URL: <https://github.com/FasterXML/jackson-databind>

Jackson is a Java library that enables the conversion of Java objects to Json (Serialization) and the conversion of Json to Java objects (Deserialization). This library is used in a lot of web applications written in Java as JSON is heavily used in REST APIs. The library is split in multiple repositories, jackson-databind contains the main serialization/deserialization logic.

Onboarding experience

Did you choose a new project or continue on the previous one? We have chosen a new one, our previous assignment involved improving tests for Mockito.

If you changed the project, how did your experience differ from before? Immediately we have the repo being able to be built without issue. The project uses a maven wrapper so it is easy to pick up.

Issue-description

The issue that can be found **here** requests a feature supporting declared `@JsonAnySetter` methods with only one parameter as shown in the example below

```
class WebhookDto {
    @JsonAnySetter
    void setUnmappedProperties(Map<String, Object> unmapped) {}
}
```

The current version only supports `@JsonAnySetter` methods with two parameters. As shown here:

```
class WebhookDto {
    @JsonAnySetter
    void setUnmapped(String key, object Val){}
}
```

`@JsonAnySetter` is a mechanism to pass unmapped key/value pairs of a Json object to a Java object. An unmapped pair is a pair that cannot be assigned to a field of the Java object. For the moment, if an unmapped pair is detected and an object contains a method that takes 2 parameters annotated with `@JsonAnySetter`, the the method is called for each pair with the key as the first argument and the value as the second.

Requirements

- If a `@JsonAnySetter` method with method signature `<method_name>(Map<String, Object>)` or `<method_name>(Map<Object, Object>)` is recognized, parsing of the JSON string should change accordingly.
- Deserializer should collect data into a `Map<String, Object>` before calling the setter method.
- Methods not fitting the method signature should throw an error.
- The extended `@JsonAnySetter` support should not conflict with previous `@JsonAnySetter` functionality.
- When multiple methods in an class are annotated with `@JsonAnySetter`, deserialization should throw an exception

- `@JsonAnySetter` can be applied to constructor arguments, class fields and methods. If multiple `JsonAnySetter` annotations are set, only one is used, in the priority order of methods then fields then constructor arguments.
- If the method is annotated with `@JsonAnySetter(enabled=false)`, then it shouldn't be called even if there unmapped values
- Inheritance should work as expected: if a method is present in the base class, it should be called with all unmapped pairs when deserializing an inheriting class.

Example :

Given this java class

```
class User {
    public String name;
    @JsonAnySetter
    void setUnknownProperties(Map<String, Object>) {}
}
```

Deserialization of the following JSON

```
{
    "name": "a name"
    "email": "an email"
    "password": "a password"
}
```

Should call `User::setUnknownProperties` with the following map :

```
Map.of(
    "email", "an email",
    "password", "a password"
)
```

Effort spent

For each team member, how much time was spent in

1. plenary discussions/meetings;

Everyone: 4 hours (thursday lab) + 2 hours (day before submission) + 2 hours (combined miscellaneous project discussions)

2. discussions within parts of the group;

Everyone: 4 hours

3. reading documentation;

- Alix: 10 min
- Leo: 30 min
- Adam: 15 min
- Anass: 30 min
- Samer: 40 min

4. configuration and setup;

Everyone: 30 min

5. analyzing code/output;

- Leo 4 hour
- Adam 4 hours
- Alix 5 hours
- Anass 3 hours
- Samer 2.5 hours

6. writing documentation;

- Alix 3 hours
- Leo 2 hours
- Adam 2 hour
- Anass 1.5 hours
- Samer 2.5 hours

7. writing code;

- Alix 5 hours
- Leo 2 hours
- Adam 2 hour
- Anass 2.5 hour
- Samer 2 hours

8. running code

Everyone ~30 min

For setting up tools and libraries (step 4), enumerate all dependencies you took care of and where you spent your time, if that time exceeds 30 minutes.

Overview of issue(s) and work done.

Title: Support @JsonAnySetter on method with single Map parameter

URL: <https://github.com/FasterXML/jackson-databind/issues/4889>

Summary in one or two sentences

Scope (functionality and code affected).

Requirements for the new feature or requirements affected by functionality being refactored

Optional (point 3): trace tests to requirements.

Code changes

Patch

1.Changes in BasicBeanDescription.java

```

289 -         if ((type != String.class) && (type != Object.class)) {
289 +         if ((type != String.class) && (type != Object.class) && (type != Map.class)) {
290     290             throw new IllegalArgumentException(String.format(
291 - "Invalid 'any-setter' annotation on method '%s()': first argument not of type String or Object, but %s",
291 + "Invalid 'any-setter' annotation on method '%s()': first argument not of type String or Object or Map, but %s",
292 + anyMethod.getName(), type.getName()));
293 +         }
294 +         if ((type == Map.class) && (anyMethod.getParameterCount() != 1)) {
295 +             throw new IllegalArgumentException(String.format(
296 + "Invalid 'any-setter' annotation on method '%s()': If method takes only one argument then it must be of type Map, but found %s",
292     297         anyMethod.getName(), type.getName()));
293     298         }
294     299         return anyMethod;

```

2.Changes in POJOPropertiesCollector.java

```

1095 1095         if (argCount == 0) { // getters (including 'any getter')
1096 1096             _addGetterMethod(props, m, _annotationIntrospector);
1097 1097         } else if (argCount == 1) { // setters
1098 -             _addSetterMethod(props, m, _annotationIntrospector);
1098 +             if (Boolean.TRUE.equals(_annotationIntrospector.hasAnySetter(m))) {
1099 +                 // If it's an @JsonAnySetter, add it to _anySetters and DO NOT process as a normal setter
1100 +                 if (_anySetters == null) {
1101 +                     _anySetters = new LinkedList<>();
1102 +                 }
1103 +                 _anySetters.add(m);
1104 +             } else {
1105 +                 // Regular setter (only if NOT annotated with @JsonAnySetter)
1106 +                 _addSetterMethod(props, m, _annotationIntrospector);
1107 +             }
1099 1108         } else if (argCount == 2) { // any setter?
1100 1109             if (Boolean.TRUE.equals(_annotationIntrospector.hasAnySetter(m))) {
1101 1110                 if (_anySetters == null) {

```

3.Changes in BeanDeserializer.java

```

278 278         }
279 279         handleUnknownVanilla(p, ctxt, bean, propName);
280 280     } while ((propName = p.nextFieldName()) != null);
281 +
282 +     // [databind#4889] This code is for 1-arg of type map anysetter methods
283 +     // Indicates that the parsing is done and that the 1-arg method can be called
284 +     // with the map of unmapped properties.
285 +     if(_anySetter != null) {
286 +         try{
287 +             _anySetter.parsingDone(bean);
288 +         }catch(Exception e){
289 +             wrapInstantiationProblem(e, ctxt);
290 +         }
291 +     }
292 +
281 293     return bean;
282 294 }
283 295
+
+
+
315 327         }
316 328         handleUnknownVanilla(p, ctxt, bean, propName);
317 329     } while ((propName = p.nextFieldName()) != null);
330 +
331 +     // [databind#4889] This code is for 1-arg of type map anysetter methods
332 +     // Indicates that the parsing is done and that the 1-arg method can be called
333 +     // with the map of unmapped properties.
334 +     if(_anySetter != null) {
335 +         try{
336 +             _anySetter.parsingDone(bean);
337 +         }catch(Exception e){
338 +             wrapInstantiationProblem(e, ctxt);
339 +         }
340 +     }
318 341     }
319 342     return bean;
320 343 }
+
+
+
405 428         }
406 429         handleUnknownVanilla(p, ctxt, bean, propName);
407 430     } while ((propName = p.nextFieldName()) != null);
408 -     // This code is for 1-arg of type map anysetter methods
431 +
432 +     // [databind#4889] This code is for 1-arg of type map anysetter methods

```

```

405 405         }
406 406 +       handleUnknownVanilla(p, ctxt, bean, propName);
407 407     } while ((propName = p.nextFieldName()) != null);
408 +
409 +     // This code is for 1-arg of type map anysetter methods
410 +     // Indicates that the parsing is done and that the 1-arg method can be called
411 +     // with the map of unmapped properties.
412 +     if(_anySetter != null) {
413 +         try{
414 +             _anySetter.parsingDone(bean);
415 +         }catch(Exception e){
416 +             wrapAndThrow(e, bean, propName, ctxt);
417 +         }
418 +     }
419
420     return bean;
421 }

```

```

411 435     if(_anySetter != null) {
412 436         try{
413 437             _anySetter.parsingDone(bean);
414 438         }catch(Exception e){
415 -             wrapAndThrow(e, bean, propName, ctxt);
416 -         }
417 -
418 +             wrapInstantiationProblem(e, ctxt);
419 +         }
420     }
421     return bean;
422 }
423
424 @@@ -721,6 +744,17 @@@ protected final Object deserializeWithView(JsonParser p, DeserializationContext
425
426 721 744     }
427 722 745     handleUnknownVanilla(p, ctxt, bean, propName);
428 723 746     } while ((propName = p.nextFieldName()) != null);
429
430 747 +
431 748 +     // [databind#4889] This code is for 1-arg of type map anysetter methods
432 749 +     // Indicates that the parsing is done and that the 1-arg method can be called
433 750 +     // with the map of unmapped properties.
434 751 +     if(_anySetter != null) {
435 752 +         try {
436 753 +             _anySetter.parsingDone(bean);
437 754 +         } catch (Exception e) {
438 755 +             wrapInstantiationProblem(e, ctxt);
439 756 +         }
440 757 +     }
441
442 724 758     }
443 725 759     return bean;
444 726 760 }

```

4. Changes in BeanDeserializerFactory.java

```

844 844         final boolean isParameter = mutator instanceof AnnotatedParameter;
845 +         boolean isMapMethod = false;
846 846         int parameterIndex = -1;
847 847
847 848 +         if (mutator instanceof AnnotatedMethod) {
848 -             // we know it's a 2-arg method, second arg is the value
849 849             AnnotatedMethod am = (AnnotatedMethod) mutator;
850 -             keyType = am.getParameterType(0);
851 -             valueType = am.getParameterType(1);
852 -             // Need to resolve for possible generic types (like Maps, Collections)
853 -             valueType = resolveMemberAndTypeAnnotations(ctxt, mutator, valueType);
854 -             prop = new BeanProperty.Std(PropertyName.construct(mutator.getName()),
855 -                 valueType, null, mutator,
856 -                 PropertyMetadata.STD_OPTIONAL);
857 +             // It can either be a 2-arg method or 1-arg of type Map.
858 +             if (am.getParameterCount() == 1) {
859 +                 JavaType paramType = am.getParameterType(0);
860 +                 paramType = resolveMemberAndTypeAnnotations(ctxt, am, paramType);
861 +                 keyType = paramType.getKeyType();
862 +                 valueType = paramType.getContentType();
863 +                 prop = new BeanProperty.Std(PropertyName.construct(mutator.getName()),
864 +                     paramType, null, mutator, PropertyMetadata.STD_OPTIONAL);
865 +                 isMapMethod = true;
866 +             }
867 +             // we know it's a 2-arg method, second arg is the value
868 +             else {
869 +                 keyType = am.getParameterType(0);
870 +                 valueType = am.getParameterType(1);
871 +                 // Need to resolve for possible generic types (like Maps, Collections)
872 +                 valueType = resolveMemberAndTypeAnnotations(ctxt, mutator, valueType);
873 +                 prop = new BeanProperty.Std(PropertyName.construct(mutator.getName()),
874 +                     valueType, null, mutator,
875 +                     PropertyMetadata.STD_OPTIONAL);
876 +             }
877
878 878         } else if (isField) {
879 879             AnnotatedField af = (AnnotatedField) mutator;
880 +             @@ -952,6 +965,9 @@ protected SettableAnyProperty constructAnySetter(DeserializationContext ctxt,
881
882 882             return SettableAnyProperty.constructForMapParameter(ctxt,
883 883                 prop, mutator, valueType, keyDeser, deser, typeDeser, parameterIndex);
884 884         }
885
886 +         if (isMapMethod) {
887 +             return SettableAnyProperty.constructForMapMethod(ctxt, prop, mutator, valueType, keyDeser, deser, typeDeser);
888 +         }
889
890 890         return SettableAnyProperty.constructForMethod(ctxt,
891 891             prop, mutator, valueType, keyDeser, deser, typeDeser);
892 892     }
893 893 }

```

5.Changes in SettableAnyProperty.java

```

88 88
89 + /**
90 +  * @since 2.10
91 +  */
92 + public static SettableAnyProperty constructForMapMethod(DeserializationContext ctxt,
93 +     BeanProperty property,
94 +     AnnotatedMember field, JavaType valueType,
95 +     KeyDeserializer keyDeser,
96 +     JsonSerializer<Object> valueDeser, TypeDeserializer typeDeser) {
97 + +     return new MapMethodAnyProperty(property, field, valueType,
98 +         keyDeser, valueDeser, typeDeser);
99 + }
100 +
101
102 101 /**
103 102  * @since 2.14
104 103  */
105 + @@ -270,6 +282,9 @@ public void set(Object instance, Object propName, Object value) throws IOExcepti
106
107 270 282
108 271     protected abstract void _set(Object instance, Object propName, Object value) throws Exception;
109 272
110 283
111 +     public void parsingDone(Object instance) throws Exception {}
112 +
113 +
114 273 288 /*

```

```

357 +
358 + /**
359 +  * @since 2.19
360 +  */
361 + protected static class MapMethodAnyProperty extends SettableAnyProperty implements java.io.Serializable {
362 +     Map<Object, Object> map = new HashMap<>();
363 +
364 +     public MapMethodAnyProperty(BeanProperty property, AnnotatedMember setter, JavaType type, KeyDeserializer keyDeser, JsonDeserializer<Object> valueDeser, TypeDeserializer typeDeser) {
365 +         super(property, setter, type, keyDeser, valueDeser, typeDeser);
366 +     }
367 +
368 +     @Override
369 +     public SettableAnyProperty withValueDeserializer(JsonDeserializer<Object> deser) {
370 +         return new MapMethodAnyProperty(_property, _setter, _type, _keyDeserializer, deser, _valueTypeDeserializer);
371 +     }
372 +
373 +     @Override
374 +     protected void _set(Object instance, Object propName, Object value) {
375 +         map.put(propName, value);
376 +     }
377 +
378 +     @Override
379 +     public void parsingDone(Object instance) throws Exception {
380 +         ((AnnotatedMethod) _setter).callOnWith(instance, map);
381 +     }
382 + }
383 +
384 + /**

```

Optional (point 4): the patch is clean.

We have made sure that there is no unnecessary whitespace or excessive code. In some spots there is commented code but we did not leave that in as it came from the original repo. Optional (point 5): considered for acceptance (passes all automated checks).

All of the tests pass, including our added tests.

Test results

The test suite includes cases that specifically verify the new feature, ensuring that it works as intended. After adding the feature, all new and existing tests pass successfully, confirming that the implementation does not introduce conflicts.

Before modification (branch 2.19)

```

[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 3552, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]

```

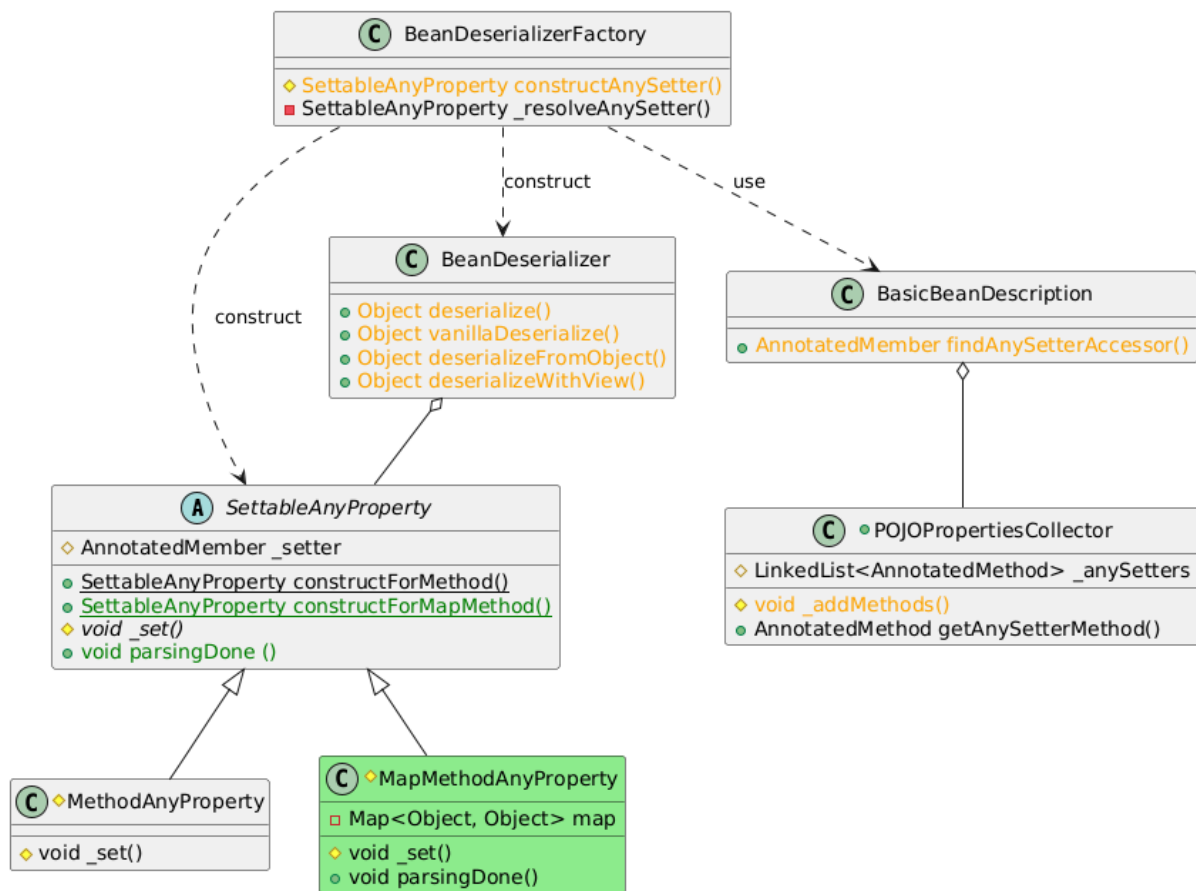
After modification (branch json-any-ampper-map-method)

```

[INFO] Results:
[INFO]
[INFO] Tests run: 3559, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]

```

UML class diagram and its description



In green are method/classes added, in orange are methods modified.

In accordance with the requirements, the goal is to : detect methods with a Map as their single parameter annotated with `@JsonAnySetter`. To do this, there are multiple classes to modify :

- `POJOPropertiesCollector._addMethods()` currently collects all the methods with two parameters that are annotated with `@JsonAnySetter`, it has been updated to also collect the annotated methods with only one parameter. This is also the class that errors out if there are multiple methods annotated.
- `BasicBeanDescription.findAnySetterAccessor()` checks the methods returned by `POJOPropertiesCollector.getAnySetterMethod()`. It checks that the methods have the correct type parameters. This has been modified to accept methods with one parameter if this parameter is a Map. This is also the methods that handles the priority between fields and methods annotated following the requirements
- `BeanDeserializerFactory` uses the Factory pattern, it is used to simplify the construction of the class that handles the main serialization logic, `BeanDeserializer`. The method `constructAnySetter` has been updated to build and return a `MapMethodAnyProperty` if the annotated member is detected to be a method with only one parameter.

The class `SettableAnyProperty` is an abstract class that models an any setter, an any setter is anything annotated with `@JsonAnySetter`, there is one subclass for each type of any setter, so for example, one for when the any setter is a Map field on the class, `MethodAnyProperty` is called when it's a method with 2 parameters that is annotated. We added a new subclass named `MapMethodAnyProperty` that is used when what's annotated is a method that takes a map as its only parameter.

When we deserialize an object, for any key/value pair in the json (called a Prop in the code), if the Prop cannot be mapped to a field of the class, then the key and value will be passed to the `_set` method of the current `SettableAnyProperty` of the object we're deserializing. For our use case, `MapMethodAnyProperty` collects these unmapped props into a Map.

To know that we have collected all unmapped pairs of the object and that we can pass it to its any setter method, we modified the `SettableAnyProperty` public API to add a `parsingDone` method that is called by `BeanDeserializer.deserialize()` (and the three other deserialization methods in `BeanDeserializer`) when the entire JSON object has been parsed (we reached a `}` in the JSON). This method `parsingDone` does nothing for all subclasses of `SettableAnyProperty` except for `MapMethodAnyProperty` where it calls the any setter method with the map of all unpaired props.

Key changes/classes affected

Optional (point 1): Architectural overview.

As given under the heading of "UML and class diagram" where we describe the internals and architecture.

Optional (point 2): relation to design pattern(s).

We haven't decided on using any new design pattern, but the code that we worked on was already using some design patterns, in particular Builder pattern and Factory pattern. Furthermore, we use as seen UML to clearly visualize our changes. We make small incremental changes using git, branching and merging to a main branch to ensure that any conflicts are resolved in piecemeal rather than large and error-prone conflict resolving.

Overall experience

What are your main take-aways from this project? What did you learn?

Learning another codebase is almost always a major commitment. When you can truly see the issues and tools available then the codebase has almost become an "acquaintance", for better and for worse (the latter mostly...). The available structure always looks a little confusing at first and even after grasping it, it might still look illogical, which is an extremely common issue in applied Computer Science. You start with a "clean" and "debtless" codebase which then gets expectations loaded onto it, then functionality has to be added in a timely manner. Suddenly the code looks almost impenetrable and refactoring becomes seemingly herculean.

How did you grow as a team, using the Essence standard to evaluate yourself

Our way of working would still be in the foundations established state. The reason for not getting to the next state is because it has not really been a priority. Our established standard has been enough for the complexity of the projects as well as the size of our team. We quickly established standards for a way of working after lab 1 that we have stuck to throughout the course. However, if the projects would grow in complexity or our team would grow, the need for more strict standards would probably lead to a migration to the next state. We do not for example regularly inspect or evaluate the satisfiability of our selected tools and established practices, which is one requirement for being considered "In Use" which is the next step in the Essence standard. Another thing we still miss out on is the lack of established method of giving feedback on our way of working. We still have the "open-chatroom" in Discord for fast messaging with communication also happening intermittently on GitHub. In conclusion there is still room for improvement.

Optional (point 6): How would you put your work in context with best software engineering practice?

We made sure to iterate on our changes incrementally. At first we were unsure of the possibility of type checking the Key of a Map type, so we put it aside for the moment and settled at checking for a Map type instead. After we had gained better insight into the capabilities of the code then we figured out how to check the Key value.

Optional (point 7): Is there something special you want to mention here?

We have done the mini-assignments so please keep that in mind if it is needed.